

Processing Workshop für Teenagers - 3 Stunden

Zeitplan

0:00 - 0:20 | Willkommen & Einführung (20 Min)

Ziel: Begeisterung wecken und Kontext schaffen

Ablauf:

- Kurze Vorstellungsrunde (5 Min)
- Präsentation: "Was ist Processing?" (10 Min)
 - Zeige inspirierende Beispiele von digitaler Kunst
 - Videos von interaktiven Installationen
 - Erkläre: "Code = kreatives Werkzeug"
- Installation & Setup überprüfen (5 Min)

Material:

- Beamer/große Bildschirm
 - Beispiel-Videos: Processing-Kunstwerke (z.B. von openprocessing.org)
 - Alle PCs haben Processing bereits installiert
-

0:20 - 1:00 | Erste Schritte - Zeichnen (40 Min)

Ziel: Sofortige visuelle Erfolge

Teil 1: Das Koordinatensystem (10 Min)

Live-Coding am Beamer:

```
size(600, 400);  
background(220);
```

- Erkläre: Canvas wie ein Blatt Papier
- Koordinaten: (0,0) ist oben links
- Zeige Sketch mit Lineal und Koordinatenanzeige
- Zeige Kreis, Rechteck, Linie, Dreieck im Sketch

Teil 2: Formen zeichnen (15 Min)

Zeige und lasse Schüler mitmachen:

```
size(600, 400);
background(220);

ellipse(300, 200, 100, 100); // Kreis
rect(100, 150, 80, 60);      // Rechteck
line(0, 0, 600, 400);        // Linie
triangle(450, 300, 500, 350, 400, 350); // Dreieck
```

Teil 3: Farben (15 Min)

```
size(600, 400);
background(0, 150, 255); // Himmelblau

fill(255, 200, 0);        // Gelb
ellipse(500, 75, 80, 80); // Sonne

fill(0, 200, 100);
noStroke();

rect(0, height/2, width, height/2); // Gras
```

Übung: "Zeichne dein Gesicht oder eine einfache Szene" (10-15 Min)

Starter-Code Eine kleines Gemälde:

```
size(600, 600);

// Himmel
background(135, 206, 235);

// Sonne
fill(255, 220, 0);
noStroke();
ellipse(500, 100, 80, 80);

// Gras
fill(34, 139, 34);
rect(0, 400, 600, 200);

// Haus
fill(139, 69, 19);
rect(200, 300, 200, 150);

// Dach
fill(178, 34, 34);
triangle(180, 300, 300, 220, 420, 300);

// Fenster
```

```
fill(173, 216, 230);
rect(240, 340, 50, 50);
rect(310, 340, 50, 50);

// Tür
fill(101, 67, 33);
rect(280, 380, 40, 70);

// Baum
fill(101, 67, 33);
rect(450, 350, 30, 100);

fill(34, 139, 34);
ellipse(465, 340, 80, 80);

// Vogel
stroke(0);
line(100, 100, 110, 110);
line(110, 110, 120, 100);

// Jetzt bist du dran!
// Ändere Farben: Rote Sonne, Blauer Baum, ...
// Füge hinzu: Wolken, Blumen, Zaun...
```

Zeige beide Beispiele kurz am Beamer, dann lassen die Schüler selbst kreativ werden!

Material:

- Handout mit Befehlen (siehe unten)
- Processing-Dateien als Vorlagen (beide Starter-Codes oben)
- Beispiel-Bilder zur Inspiration (optional)

1:00 - 1:10 | Kurze Pause (10 Min)

1:10 - 1:50 | Interaktivität & Animation (40 Min)

Ziel: Programme lebendig machen

Teil 1: setup() und draw() (10 Min)

```
void setup() {
  size(600, 400);
  background(255);
}

void draw() {
```

```
ellipse(mouseX, mouseY, 50, 50);  
}
```

- Erkläre: setup läuft 1x, draw läuft wiederholt
- Zeige: Maus-Zeichenprogramm

Teil 2: Variablen & Bewegung (15 Min)

```
float x = 0;  
  
void setup() {  
  size(600, 400);  
}  
  
void draw() {  
  background(220);  
  
  ellipse(x, 200, 50, 50);  
  x = x + 2;  
  
  if (x > width) {  
    x = 0;  
  }  
}
```

```
void setup() {  
  size(600, 400);  
  background(220);  
}  
  
void draw() {  
  
  ellipse(random(600), random(200), 50, 50);  
}
```

Teil 3: Maus-Interaktion (15 Min)

```
void setup() {  
  size(600, 400);  
  background(255);  
}  
  
void draw() {  
  if (mousePressed) {  
    fill(random(255), random(255), random(255));  
    ellipse(mouseX, mouseY, 30, 30);  
  }  
}
```

```
}  
}
```

Übung: "Erstelle ein interaktives Zeichenprogramm mit verschiedenen Farben"

Material:

- Code-Beispiele zum Copy-Paste
 - Cheat Sheet mit mouseX, mouseY, mousePressed
-

1:50 - 2:40 | Kreatives Projekt (50 Min)

Ziel: Eigene Ideen umsetzen

Projektauswahl (5 Min)

Schüler wählen eines:

1. **Interaktives Zeichenprogramm** (leicht)
2. **Einfaches Spiel** - Ball fangen (mittel)
3. **Generative Kunst** - Muster, die sich verändern (mittel)
4. **Eigene Idee** (mit Unterstützung)

Arbeitszeit (35 Min)

- Schüler arbeiten selbstständig
- Du gehst herum und hilfst
- Ermutige Experimente

Abschluss (10 Min)

- Code speichern
- Screenshots machen
- Projekt vorbereiten zum Zeigen

Material:

- Projekt-Templates (siehe unten)
 - Hilfe-Karten für häufige Probleme
 - USB-Sticks zum Speichern
-

2:40 - 3:00 | Präsentation & Abschluss (20 Min)

Präsentationen (15 Min)

- Jede/r zeigt kurz ihr/sein Projekt (1-2 Min pro Person)
- Publikum gibt positives Feedback

- Du zeigst Wertschätzung für alle Ansätze

Ausblick (5 Min)

- Wo können sie weitermachen?
 - openprocessing.org (Projekte hochladen & teilen)
 - The Coding Train auf YouTube
 - processing.org/tutorials
- Feedback-Runde: Was hat Spaß gemacht?

Material:

- Feedback-Zettel (optional)
 - Liste mit Ressourcen zum Mitnehmen
-

Benötigtes Material

Technik

- Computer für jeden Teilnehmer (Processing installiert)
- Beamer/großer Bildschirm
- Lautsprecher (für Beispiel-Videos)
- Maus für jeden PC empfohlen

Handouts

1. **Befehls-Cheatsheet** (1 Seite)
2. **Projekt-Templates** (Code zum Starten)
3. **Ressourcen-Liste** zum Mitnehmen

Vorbereitung

- Processing auf allen PCs installieren und testen
 - Beispiel-Dateien auf allen PCs verfügbar machen
 - Ordner für Schüler-Projekte erstellen
 - Backup-USB-Sticks bereithalten
-

Projekt-Templates

Template 1: Zeichen-App

```
void setup() {  
  size(800, 600);  
  background(255);  
}  
  
void draw() {
```

```
if (mousePressed) {  
  if (mouseButton == LEFT) {  
    fill(0);  
    ellipse(mouseX, mouseY, 20, 20);  
  } else if (mouseButton == RIGHT) {  
    fill(255);  
    ellipse(mouseX, mouseY, 40, 40);  
  }  
}  
}  
  
void keyPressed() {  
  if (key == 'c') {  
    background(255);  
  }  
}
```

Template 2: Ball fangen

```
float ballX, ballY;  
float ballSpeedX = 3;  
float ballSpeedY = 3;  
int score = 0;  
  
void setup() {  
  size(600, 400);  
  ballX = width/2;  
  ballY = height/2;  
}  
  
void draw() {  
  background(200);  
  
  // Ball bewegen  
  ballX += ballSpeedX;  
  ballY += ballSpeedY;  
  
  // Ball abprallen  
  if (ballX > width || ballX < 0) {  
    ballSpeedX *= -1;  
  }  
  if (ballY > height || ballY < 0) {  
    ballSpeedY *= -1;  
  }  
  
  // Ball zeichnen  
  fill(255, 0, 0);  
  ellipse(ballX, ballY, 30, 30);  
  
  // Fänger (Maus)  
  fill(0, 0, 255);
```

```
rect(mouseX-40, height-30, 80, 20);

// Score
fill(0);
textSize(20);
text("Score: " + score, 10, 25);
}
```

Template 3: Generative Kunst

```
void setup() {
  size(600, 600);
  background(0);
}

void draw() {
  // Transparenter Hintergrund für Trail-Effekt
  fill(0, 5);
  rect(0, 0, width, height);

  // Zufällige Kreise
  fill(random(100, 255), random(100, 255), random(100, 255), 150);
  noStroke();
  float x = width/2 + cos(frameCount * 0.05) * 200;
  float y = height/2 + sin(frameCount * 0.05) * 200;
  ellipse(x, y, random(20, 50), random(20, 50));
}

void mousePressed() {
  background(0);
}
```

Tipps für den Unterricht

Dos

- **Zeige zuerst, dann lassen sie machen** - Live-Coding ist wichtig
- **Ermutige Experimente** - "Was passiert wenn...?"
- **Feiere Fehler** - Bugs sind Lernchancen
- **Gehe herum** - Individuelle Hilfe ist Gold wert
- **Nutze visuelle Beispiele** - Zeige coole Processing-Kunst

Don'ts

- Nicht zu viel Theorie - Teenager wollen machen
- Nicht zu schnell - Warte bis alle mitkommen
- Nicht perfekt sein müssen - Hauptsache es macht Spaß
- Keine komplizierten Programmierkonzepte - OOP, Arrays etc. weglassen

Bei Problemen

- **"Es funktioniert nicht"** → Zeige Fehlersuche (Semikolons, Klammern)
 - **"Ich weiß nicht was ich machen soll"** → Stelle Fragen, zeige Beispiele
 - **"Das ist zu schwer"** → Vereinfache, ermutige kleine Schritte
 - **Zu große Unterschiede im Tempo** → Habe Extra-Challenges bereit
-

Erfolgsmetriken

Am Ende sollten die Teilnehmer:

- ✓ Ein funktionierendes Programm geschrieben haben
- ✓ Verstehen, dass Code kreativ sein kann
- ✓ Grundlegende Konzepte kennen (Koordinaten, Farben, Interaktion)
- ✓ Spaß gehabt haben und motiviert sein weiterzumachen